

LocalAudit AI — Production SaaS · Global Architecture

Full Codex Build Prompt (Senior Engineer Level)

Paste this entire file into Codex. Build one service at a time.
This is a production-grade, globally deployable, multi-tenant SaaS.

SYSTEM OVERVIEW

Product: LocalAudit AI — automated website audit + AI-fix delivery SaaS **Model:** Multi-tenant subscription SaaS with Stripe billing **Global reach:** Works in any country. Stripe handles 135+ currencies. GDPR + PCI-DSS compliant. **Architecture:** Microservices, async job queue, Cloudflare edge, Supabase Postgres

TECH STACK (production-grade, mostly free tiers)

Layer	Technology	Why
Frontend	Next.js 14 (App Router) on Vercel	Edge rendering, ISR, global CDN
API	FastAPI (Python 3.12) on Railway	Async, OpenAPI docs, type-safe
Auth	Supabase Auth	Row-level security, OAuth, JWT
Database	Supabase PostgreSQL	Managed, row-level security, realtime
Cache	Upstash Redis	Serverless Redis, free tier
Queue	BullMQ (Node.js worker)	Reliable job queuing on Redis
Browser	Playwright in Docker	Headless Chromium, scalable
AI	Anthropic Claude API	Haiku (fast) for audit, Sonnet for fix
Payment	Stripe	Subscriptions, webhooks, Customer Portal
File storage	Cloudflare R2	S3-compatible, free egress
Email	Resend	Transactional email, DKIM
Edge/CDN	Cloudflare	WAF, DDoS, rate limiting, TLS
Monitoring	Sentry + Grafana Cloud	Error tracking + metrics
CI/CD	GitHub Actions	Auto-deploy on push

REPO STRUCTURE

```
localaudit-saas/
├─ apps/
│   ├─ web/                                # Next.js 14 frontend
│   │   ├─ app/
│   │   │   ├─ (marketing)/                # Public landing, pricing pages
│   │   │   ├─ (dashboard)/                # Protected dashboard routes
│   │   │   ├─ api/
│   │   │   │   └─ webhooks/
│   │   │   │       └─ stripe/             # Stripe webhook endpoint
│   │   │   └─ layout.tsx
│   │   ├─ components/
│   │   └─ lib/
│   │       ├─ stripe.ts
│   │       └─ supabase.ts
│   └─ middleware.ts                        # Auth guard
├─ api/                                    # FastAPI backend
│   ├─ main.py
│   ├─ routers/
│   │   ├─ audits.py
│   │   ├─ tenants.py
│   │   ├─ billing.py
│   │   └─ health.py
│   ├─ services/
│   │   ├─ audit_service.py
│   │   ├─ ai_service.py
│   │   ├─ scraper_service.py
│   │   └─ delivery_service.py
│   ├─ models/                             # SQLAlchemy models
│   ├─ schemas/                             # Pydantic schemas
│   └─ core/
│       ├─ config.py
│       ├─ database.py
│       └─ security.py
├─ workers/                                # BullMQ job workers (Node.js)
│   └─ src/
│       ├─ index.ts                         # Worker entry point
│       └─ queues/
│           ├─ audit.queue.ts
│           ├─ fix.queue.ts
│           └─ delivery.queue.ts
```

```
| | | └─ processors/
| | |   └─ audit.processor.ts
| | |   └─ fix.processor.ts
| | |   └─ scraper.processor.ts
| | └─ Dockerfile
| └─ package.json
|
└─ infra/
  └─ cloudflare/
    └─ wrangler.toml      # Cloudflare Workers config
    └─ docker-compose.yml # Local dev environment
  └─ supabase/
    └─ migrations/        # Database schema migrations
  └─ .github/
    └─ workflows/
      └─ deploy.yml
```

DATABASE SCHEMA (Supabase PostgreSQL)

sql

```

-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- Plans configuration
CREATE TABLE plans (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name TEXT NOT NULL,                                -- 'starter', 'pro', 'agency'
  display_name TEXT NOT NULL,
  stripe_price_id TEXT NOT NULL UNIQUE,              -- Stripe Price ID
  stripe_price_id_annual TEXT,                       -- Annual billing option
  monthly_credits INTEGER NOT NULL,                 -- Audits allowed per month
  max_users INTEGER DEFAULT 1,
  features JSONB DEFAULT '{}',                      -- Feature flags
  price_usd INTEGER NOT NULL,                      -- Cents (e.g. 2900 = $29)
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Tenants (companies/individuals who pay)
CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,                        -- URL-safe identifier
  plan_id UUID REFERENCES plans(id),
  stripe_customer_id TEXT UNIQUE,                   -- Stripe Customer ID
  stripe_subscription_id TEXT UNIQUE,               -- Stripe Subscription ID
  subscription_status TEXT DEFAULT 'inactive',      -- active|past_due|canceled|trialing
  credits_used INTEGER DEFAULT 0,
  credits_limit INTEGER DEFAULT 0,
  billing_cycle_start TIMESTAMPTZ,
  billing_cycle_end TIMESTAMPTZ,
  trial_ends_at TIMESTAMPTZ,
  country TEXT,                                     -- ISO 3166-1 alpha-2
  currency TEXT DEFAULT 'usd',                     -- ISO 4217
  timezone TEXT DEFAULT 'UTC',
  gdpr_consented_at TIMESTAMPTZ,                   -- GDPR compliance
  deletion_requested_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Users (members of a tenant)
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  auth_user_id UUID UNIQUE NOT NULL,               -- Supabase Auth user ID

```

```

email TEXT NOT NULL,
full_name TEXT,
role TEXT DEFAULT 'member',          -- owner|admin|member
is_active BOOLEAN DEFAULT true,
last_login TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- API Keys (for programmatic access)
CREATE TABLE api_keys (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  name TEXT NOT NULL,                -- Human label
  key_prefix TEXT NOT NULL,          -- First 8 chars (for display)
  key_hash TEXT NOT NULL UNIQUE,     -- SHA-256 of full key
  scopes TEXT[] DEFAULT '{"audit:read","audit:create"}',
  last_used_at TIMESTAMPTZ,
  expires_at TIMESTAMPTZ,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Audit jobs
CREATE TABLE audits (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  created_by UUID REFERENCES users(id),
  business_name TEXT,
  website_url TEXT NOT NULL,
  niche TEXT,
  city TEXT,
  country TEXT,
  status TEXT DEFAULT 'pending',     -- pending|running|completed|failed
  issues JSONB DEFAULT '[]',        -- Array of issue objects
  score INTEGER,                    -- 0-100
  load_time_ms INTEGER,
  screenshot_before TEXT,           -- R2 URL
  error TEXT,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Fix jobs (one per issue per audit)
CREATE TABLE fixes (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  audit_id UUID NOT NULL REFERENCES audits(id) ON DELETE CASCADE,

```

```

tenant_id UUID NOT NULL REFERENCES tenants(id),
issue_type TEXT NOT NULL, -- 'missing_meta', 'no_ssl', etc.
issue_description TEXT,
patch_html TEXT, -- Generated HTML/CSS patch
patch_instructions TEXT, -- Plain-language guide
status TEXT DEFAULT 'pending', -- pending|generated|tested|delivered
test_passed BOOLEAN,
screenshot_after TEXT, -- R2 URL
ai_model TEXT, -- Which Claude model was used
ai_tokens_used INTEGER,
attempts INTEGER DEFAULT 0,
error TEXT,
generated_at TIMESTAMPTZ,
tested_at TIMESTAMPTZ,
delivered_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Delivery packages
CREATE TABLE deliveries (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  audit_id UUID NOT NULL REFERENCES audits(id),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  client_email TEXT NOT NULL,
  zip_file_url TEXT, -- R2 URL to delivery ZIP
  report_pdf_url TEXT, -- R2 URL to PDF report
  status TEXT DEFAULT 'pending', -- pending|sent|opened|replied
  sent_at TIMESTAMPTZ,
  opened_at TIMESTAMPTZ,
  replied_at TIMESTAMPTZ,
  follow_up_1_at TIMESTAMPTZ,
  follow_up_2_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Stripe events log (idempotency)
CREATE TABLE stripe_events (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  stripe_event_id TEXT UNIQUE NOT NULL, -- Stripe event ID
  event_type TEXT NOT NULL,
  payload JSONB NOT NULL,
  processed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Usage tracking
CREATE TABLE usage_logs (

```

```

id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id UUID NOT NULL REFERENCES tenants(id),
action TEXT NOT NULL, -- 'audit_created', 'fix_generated'
credits_consumed INTEGER DEFAULT 1,
metadata JSONB DEFAULT '{}',
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Row-level security policies
ALTER TABLE tenants ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE audits ENABLE ROW LEVEL SECURITY;
ALTER TABLE fixes ENABLE ROW LEVEL SECURITY;
ALTER TABLE deliveries ENABLE ROW LEVEL SECURITY;

-- Tenants can only see their own data
CREATE POLICY "tenant_isolation" ON audits
  FOR ALL USING (tenant_id = auth.jwt() ->> 'tenant_id');

-- Indexes
CREATE INDEX idx_audits_tenant ON audits(tenant_id, created_at DESC);
CREATE INDEX idx_fixes_audit ON fixes(audit_id);
CREATE INDEX idx_stripe_events ON stripe_events(stripe_event_id);
CREATE INDEX idx_api_keys_hash ON api_keys(key_hash);

```

MODULE 1 — Stripe Integration (Next.js)

apps/web/lib/stripe.ts

typescript

```

import Stripe from 'stripe';

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2024-06-20',
  typescript: true,
});

// Create a checkout session for a new subscription
export async function createCheckoutSession({
  priceId,
  tenantId,
  userId,
  customerEmail,
  successUrl,
  cancelUrl,
  currency = 'usd',
}: {
  priceId: string;
  tenantId: string;
  userId: string;
  customerEmail: string;
  successUrl: string;
  cancelUrl: string;
  currency?: string;
}) {
  const session = await stripe.checkout.sessions.create({
    mode: 'subscription',
    payment_method_types: ['card'],
    line_items: [{ price: priceId, quantity: 1 }],
    success_url: `${successUrl}?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: cancelUrl,
    customer_email: customerEmail,
    currency,
    allow_promotion_codes: true,
    subscription_data: {
      trial_period_days: 14, // 14-day free trial
      metadata: { tenant_id: tenantId, user_id: userId },
    },
    metadata: { tenant_id: tenantId, user_id: userId },
    automatic_tax: { enabled: true }, // Stripe Tax for global VAT/GST
    tax_id_collection: { enabled: true }, // Collect VAT numbers (B2B)
  });
  return session;
}

// Create Stripe Customer Portal session (self-service billing management)

```



```
export async function createPortalSession(stripeCustomerId: string, returnUrl: string) {
  return stripe.billingPortal.sessions.create({
    customer: stripeCustomerId,
    return_url: returnUrl,
  });
}

// Construct verified webhook event
export function constructWebhookEvent(payload: string | Buffer, signature: string) {
  return stripe.webhooks.constructEvent(
    payload,
    signature,
    process.env.STRIPE_WEBHOOK_SECRET!
  );
}
```

apps/web/app/api/webhooks/stripe/route.ts

typescript

```
import { NextRequest, NextResponse } from 'next/server';
import { constructWebhookEvent } from '@lib/stripe';
import { createServerClient } from '@supabase/ssr';

// CRITICAL: Disable body parsing (Stripe needs raw body for signature verification)
export const runtime = 'nodejs';

export async function POST(req: NextRequest) {
  const signature = req.headers.get('stripe-signature');
  if (!signature) return NextResponse.json({ error: 'No signature' }, { status: 400 });

  const payload = await req.text();
  let event: Stripe.Event;

  try {
    event = constructWebhookEvent(payload, signature);
  } catch (err) {
    return NextResponse.json({ error: 'Invalid signature' }, { status: 400 });
  }

  const supabase = createServerClient(/* ... */);

  // Idempotency: check if this event was already processed
  const { data: existing } = await supabase
    .from('stripe_events')
    .select('id')
    .eq('stripe_event_id', event.id)
    .single();

  if (existing) {
    return NextResponse.json({ status: 'already_processed' });
  }

  // Log the event immediately
  await supabase.from('stripe_events').insert({
    stripe_event_id: event.id,
    event_type: event.type,
    payload: event,
  });

  // Handle each event type
  switch (event.type) {
    case 'checkout.session.completed': {
      const session = event.data.object as Stripe.Checkout.Session;
      if (session.mode !== 'subscription') break;
    }
  }
}
```

```

const tenantId = session.metadata?.tenant_id;
const subscriptionId = session.subscription as string;

// Fetch subscription details from Stripe
const subscription = await stripe.subscriptions.retrieve(subscriptionId);
const priceId = subscription.items.data[0].price.id;

// Find the plan matching this price
const { data: plan } = await supabase
  .from('plans')
  .select('*')
  .eq('stripe_price_id', priceId)
  .single();

// Activate tenant subscription
await supabase.from('tenants').update({
  stripe_customer_id: session.customer as string,
  stripe_subscription_id: subscriptionId,
  plan_id: plan?.id,
  subscription_status: 'active',
  credits_limit: plan?.monthly_credits || 50,
  credits_used: 0,
  billing_cycle_start: new Date(subscription.current_period_start * 1000).toISOString(),
  billing_cycle_end: new Date(subscription.current_period_end * 1000).toISOString(),
  currency: session.currency || 'usd',
}).eq('id', tenantId);

// Generate and send API key
await generateAndSendApiKey(tenantId, supabase);
// Send welcome email
await sendWelcomeEmail(session.customer_email!, plan?.display_name);
break;
}

case 'invoice.payment_succeeded': {
  // Monthly renewal – reset credits
  const invoice = event.data.object as Stripe.Invoice;
  const subscriptionId = invoice.subscription as string;

  const { data: tenant } = await supabase
    .from('tenants')
    .select('*')
    .eq('stripe_subscription_id', subscriptionId)
    .single();

  if (tenant) {
    await supabase.from('tenants').update({

```

```

        credits_used: 0,
        subscription_status: 'active',
        billing_cycle_start: new Date().toISOString(),
    }).eq('id', tenant.id);
}
break;
}

case 'customer.subscription.deleted': {
    const subscription = event.data.object as Stripe.Subscription;
    await supabase.from('tenants')
        .update({ subscription_status: 'canceled', credits_limit: 0 })
        .eq('stripe_subscription_id', subscription.id);
    break;
}

case 'invoice.payment_failed': {
    const invoice = event.data.object as Stripe.Invoice;
    const subscription = event.data.object as any;
    await supabase.from('tenants')
        .update({ subscription_status: 'past_due' })
        .eq('stripe_subscription_id', invoice.subscription);
    // Send payment failed email
    break;
}
}

// Mark event as processed
await supabase.from('stripe_events')
    .update({ processed_at: new Date().toISOString() })
    .eq('stripe_event_id', event.id);

return NextResponse.json({ received: true });
}

```

MODULE 2 — Auth & Multi-tenancy Middleware

apps/web/middleware.ts

typescript

```

import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { createServerClient } from '@supabase/ssr';

export async function middleware(request: NextRequest) {
  const response = NextResponse.next();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { /* cookie handlers */ } }
  );

  const { data: { session } } = await supabase.auth.getSession();

  // Protect dashboard routes
  if (request.nextUrl.pathname.startsWith('/dashboard')) {
    if (!session) {
      return NextResponse.redirect(new URL('/login', request.url));
    }

    // Check subscription status
    const { data: user } = await supabase
      .from('users')
      .select('tenant:tenants(subscription_status, credits_used, credits_limit)')
      .eq('auth_user_id', session.user.id)
      .single();

    const tenant = user?.tenant as any;

    if (tenant?.subscription_status === 'canceled') {
      return NextResponse.redirect(new URL('/billing/reactivate', request.url));
    }
  }

  return response;
}

export const config = {
  matcher: ['/dashboard/:path*', '/api/:path*'],
};

```

MODULE 3 — FastAPI Backend Core

`apps/api/core/config.py`

python

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    # Database
    DATABASE_URL: str
    REDIS_URL: str

    # Auth
    SUPABASE_URL: str
    SUPABASE_SERVICE_KEY: str
    JWT_SECRET: str

    # AI
    ANTHROPIC_API_KEY: str
    CLAUDE_AUDIT_MODEL: str = "claude-haiku-4-5-20251001" # Fast + cheap for audit
    CLAUDE_FIX_MODEL: str = "claude-sonnet-4-20250514" # Smarter for code fixes

    # Storage
    CLOUDFLARE_R2_ACCOUNT_ID: str
    CLOUDFLARE_R2_ACCESS_KEY: str
    CLOUDFLARE_R2_SECRET_KEY: str
    CLOUDFLARE_R2_BUCKET: str
    CLOUDFLARE_R2_PUBLIC_URL: str

    # Email
    RESEND_API_KEY: str
    FROM_EMAIL: str = "noreply@localauditai.com"

    # Rate limiting
    RATE_LIMIT_PER_MINUTE: int = 60
    MAX_CONCURRENT_SCRAPES: int = 5

    # Plans
    STARTER_CREDITS: int = 50
    PRO_CREDITS: int = 200
    AGENCY_CREDITS: int = 99999

    class Config:
        env_file = ".env"

@lru_cache()
def get_settings():
    return Settings()
```

```
python
```



```

from fastapi import APIRouter, Depends, HTTPException, BackgroundTasks
from typing import Optional
from uuid import UUID
import httpx

from ..core.database import get_db
from ..core.security import get_current_tenant, check_credits
from ..schemas.audits import AuditCreate, AuditResponse
from ..services.queue_service import enqueue_audit

router = APIRouter(prefix="/audits", tags=["audits"])

@router.post("/", response_model=AuditResponse, status_code=202)
async def create_audit(
    payload: AuditCreate,
    tenant=Depends(get_current_tenant),
    db=Depends(get_db),
):
    """Create an audit job. Checks credits, enqueues async job."""
    # Verify credits available
    if not await check_credits(tenant, db):
        raise HTTPException(429, "Credit limit reached. Upgrade your plan.")

    # Create audit record
    audit = await db.fetch_one("""
        INSERT INTO audits (tenant_id, website_url, business_name, niche, city, status)
        VALUES ($1, $2, $3, $4, $5, 'pending')
        RETURNING *
    """, tenant.id, str(payload.website_url), payload.business_name,
        payload.niche, payload.city)

    # Deduct credit immediately
    await db.execute("""
        UPDATE tenants SET credits_used = credits_used + 1 WHERE id = $1
    """, tenant.id)

    # Enqueue the audit job
    await enqueue_audit(str(audit['id']), str(tenant.id), str(payload.website_url))

    return AuditResponse(**dict(audit))

@router.get("/{audit_id}", response_model=AuditResponse)
async def get_audit(
    audit_id: UUID,
    tenant=Depends(get_current_tenant),

```

```

        db=Depends(get_db),
    ):
        audit = await db.fetch_one(
            "SELECT * FROM audits WHERE id = $1 AND tenant_id = $2",
            audit_id, tenant.id
        )
        if not audit:
            raise HTTPException(404, "Audit not found")
        return AuditResponse(**dict(audit))

@router.post("/{audit_id}/fix", status_code=202)
async def request_fix(
    audit_id: UUID,
    tenant=Depends(get_current_tenant),
    db=Depends(get_db),
):
    """Client approves fix – enqueue fix agent jobs."""
    audit = await db.fetch_one(
        "SELECT * FROM audits WHERE id = $1 AND tenant_id = $2 AND status = 'completed'",
        audit_id, tenant.id
    )
    if not audit:
        raise HTTPException(404, "Audit not found or not completed")

    # Enqueue fix jobs for each issue found
    issues = audit['issues']
    for issue in issues:
        await enqueue_fix(str(audit_id), str(tenant.id), issue)

    return {"message": f"Fix jobs enqueued for {len(issues)} issues", "audit_id": str(audit_id)}

```

MODULE 4 — AI Services (Python)

`apps/api/services/ai_service.py`

python

```

import anthropic
import json
from typing import Optional
from ..core.config import get_settings

settings = get_settings()
client = anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

async def generate_audit_report(
    business_name: str,
    website: str,
    city: str,
    issues: list[dict],
) -> str:
    """Generate personalized audit report using Claude Haiku."""
    issue_list = "\n".join(f"- {issue['name']}: {issue['description']}" for issue in issues)

    response = client.messages.create(
        model=settings.CLAUDE_AUDIT_MODEL,
        max_tokens=500,
        system="""You are a friendly web consultant. Write a short, personalized audit summary for a local business. Be specific, warm, professional – not salesy. Under 200 words. Plain text only. End with one clear call to action.""",
        messages=[{
            "role": "user",
            "content": f"""Business: {business_name}
City: {city}
Website: {website}
Issues found:
{issue_list}

Write a warm, personalized audit summary that shows you actually looked at their site"""
        }]
    )
    return response.content[0].text

async def generate_html_fix(
    website_html: str,
    issue_type: str,
    issue_description: str,
    business_info: dict,
) -> dict:
    """Generate a targeted HTML/CSS fix for one specific issue using Claude Sonnet."""

```

```
FIXABLE_ISSUES = {
    "missing_viewport": "Add mobile viewport meta tag",
    "missing_meta_description": "Add meta description tag",
    "missing_meta_title": "Improve title tag",
    "no_whatsapp_button": "Add floating WhatsApp contact button",
    "no_google_maps_embed": "Add Google Maps embed",
    "missing_contact_form": "Add simple contact form",
    "no_social_links": "Add social media links to footer",
    "outdated_copyright": "Update copyright year in footer",
    "no_phone_display": "Add visible phone number",
    "missing_favicon": "Add favicon link tag",
    "missing_og_tags": "Add Open Graph meta tags for social sharing",
    "missing_schema_markup": "Add basic LocalBusiness schema.org markup",
}
```

```
system_prompt = f"""You are a senior web developer. Given HTML source code, generate a minimal, surgical patch to fix ONE specific issue: {FIXABLE_ISSUES.get(issue_type, "missing meta tags")}.

Rules:
```

- Generate the minimal HTML/CSS change needed – do not rewrite the whole page
- Output ONLY valid JSON with this structure:

```
{
  "patch_type": "inject_head"|"inject_body_end"|"replace_pattern",
  "patch_code": "the actual HTML/CSS to inject",
  "replace_pattern": "original string" (only if patch_type is replace_pattern),
  "instructions": "plain-English guide for the client to upload this change",
  "estimated_impact": "one sentence on why this helps their business"
}
```
- patch_code must be valid, production-ready HTML
- Use the business info to personalize (e.g. actual phone number in schema markup)
- Never output anything except the JSON object"""

```
response = client.messages.create(
    model=settings.CLAUDE_FIX_MODEL,
    max_tokens=2000,
    system=system_prompt,
    messages=[{
        "role": "user",
        "content": f"""Issue to fix: {issue_type}
Business info: {json.dumps(business_info)}

Current HTML (first 8000 chars):
{website_html[:8000]}

Generate the fix."""
    }])
```

```
# Parse JSON response safely
```

```

text = response.content[0].text
# Strip possible markdown code fences
text = text.strip().removeprefix("```json").removeprefix("```").removesuffix("```")
return json.loads(text)

async def validate_fix_with_vision(
    screenshot_before: bytes,
    screenshot_after: bytes,
    issue_type: str,
) -> dict:
    """Use Claude's vision to validate the fix looks correct."""
    import base64

    response = client.messages.create(
        model=settings.CLAUDE_FIX_MODEL,
        max_tokens=300,
        messages=[{
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": f"I applied a fix for '{issue_type}'. Compare these two
                },
                {
                    "type": "image",
                    "source": {"type": "base64", "media_type": "image/png",
                        "data": base64.b64encode(screenshot_before).decode()}
                },
                {
                    "type": "image",
                    "source": {"type": "base64", "media_type": "image/png",
                        "data": base64.b64encode(screenshot_after).decode()}
                }
            ],
        }
    ]
    )
    text = response.content[0].text.strip().removeprefix("```json").removesuffix("```")
    return json.loads(text)

```

MODULE 5 — Scraper Service (Python)

apps/api/services/scraper_service.py

```
python
```

```

import asyncio
import time
import re
import ssl
import socket
import requests
from bs4 import BeautifulSoup
from playwright.async_api import async_playwright
from typing import Optional
from datetime import datetime

ISSUES_CONFIG = {
    "no_https": {"name": "No HTTPS / SSL", "score_penalty": 20,
                  "description": "Website uses insecure HTTP – browsers warn visitors"},
    "site_unreachable": {"name": "Website unreachable", "score_penalty": 30,
                          "description": "Could not load the website"},
    "slow_load": {"name": "Slow load time (>4s)", "score_penalty": 12,
                   "description": "Page loads slowly – Google penalizes this in search"},
    "missing_viewport": {"name": "Not mobile-friendly", "score_penalty": 15,
                          "description": "Missing viewport meta tag – broken on mobile"},
    "missing_meta_title": {"name": "Missing title tag", "score_penalty": 8,
                            "description": "No or empty title tag – invisible to search engines"},
    "missing_meta_description": {"name": "Missing meta description", "score_penalty": 8,
                                  "description": "No description for search engine results"},
    "no_whatsapp_button": {"name": "No WhatsApp button", "score_penalty": 10,
                            "description": "Missing WhatsApp link – most customers prefer WhatsApp"},
    "no_google_maps_embed": {"name": "No Google Maps", "score_penalty": 8,
                              "description": "No map embed – harder for customers to find location"},
    "no_social_links": {"name": "No social media links", "score_penalty": 5,
                         "description": "No Facebook/Instagram links – missed trust and engagement"},
    "no_contact_form": {"name": "No contact form", "score_penalty": 7,
                         "description": "No online inquiry form – losing leads"},
    "missing_phone": {"name": "No phone number visible", "score_penalty": 6,
                       "description": "Phone number not visible – customers can't call"},
    "outdated_copyright": {"name": "Outdated copyright year", "score_penalty": 4,
                            "description": "Footer shows old year – looks abandoned"},
    "missing_og_tags": {"name": "No social sharing meta tags", "score_penalty": 4,
                         "description": "Looks broken when shared on Facebook/WhatsApp"},
    "missing_schema": {"name": "No LocalBusiness schema", "score_penalty": 5,
                        "description": "Missing schema.org markup – less rich search results"}
}

async def analyze_website(url: str) -> dict:
    """Full website audit. Returns issues, score, and screenshot."""

```

```
issues_found = []
reachable = True
html = ""
load_time_ms = 0

# Ensure HTTPS version is tested
if not url.startswith("http"):
    url = "https://" + url

# Check 1: HTTPS
if not url.startswith("https://"):
    issues_found.append("no_https")

# Check 2: Reachability + load time
try:
    start = time.time()
    response = requests.get(url, timeout=10, headers={
        "User-Agent": "LocalAuditBot/1.0 (+https://localauditai.com)"
    }, allow_redirects=True)
    load_time_ms = int((time.time() - start) * 1000)
    html = response.text

    if response.status_code >= 400:
        issues_found.append("site_unreachable")
        reachable = False
except Exception:
    issues_found.append("site_unreachable")
    reachable = False

if not reachable:
    return {
        "reachable": False,
        "issues": [{"id": i, **ISSUES_CONFIG[i]} for i in issues_found],
        "score": 0,
        "load_time_ms": 0,
        "html": "",
        "screenshot": None,
    }

# Check 3: Load time
if load_time_ms > 4000:
    issues_found.append("slow_load")

# Parse HTML
soup = BeautifulSoup(html, "html.parser")

# Check 4: Mobile viewport
```



```
viewport = soup.find("meta", attrs={"name": "viewport"})
if not viewport:
    issues_found.append("missing_viewport")

# Check 5: Title tag
title = soup.find("title")
if not title or not title.get_text(strip=True):
    issues_found.append("missing_meta_title")

# Check 6: Meta description
meta_desc = soup.find("meta", attrs={"name": "description"})
if not meta_desc or not meta_desc.get("content", "").strip():
    issues_found.append("missing_meta_description")

# Check 7: WhatsApp
html_lower = html.lower()
if "wa.me" not in html_lower and "whatsapp" not in html_lower:
    issues_found.append("no_whatsapp_button")

# Check 8: Google Maps embed
if "maps.google" not in html_lower and "goo.gl/maps" not in html_lower and "maps"
    issues_found.append("no_google_maps_embed")

# Check 9: Social links
social_patterns = ["facebook.com", "instagram.com", "twitter.com", "linkedin.com"]
if not any(p in html_lower for p in social_patterns):
    issues_found.append("no_social_links")

# Check 10: Contact form
forms = soup.find_all("form")
has_contact_form = any(
    form.find("input", {"type": ["email", "text"]}) for form in forms
)
if not has_contact_form:
    issues_found.append("no_contact_form")

# Check 11: Phone number
phone_pattern = r"[\+]?([]?[0-9]{3}[ ])?[-\s\.]?[0-9]{3}[-\s\.]?[0-9]{4,6}"
if not re.search(phone_pattern, html):
    issues_found.append("missing_phone")

# Check 12: Copyright year
current_year = datetime.now().year
copyright_pattern = r"©\s*(\d{4})"
match = re.search(copyright_pattern, html)
if match and int(match.group(1)) < current_year - 1:
    issues_found.append("outdated_copyright")
```

```

# Check 13: Open Graph tags
og_title = soup.find("meta", property="og:title")
if not og_title:
    issues_found.append("missing_og_tags")

# Check 14: Schema.org LocalBusiness
if "localbusiness" not in html_lower and "application/ld+json" not in html_lower:
    issues_found.append("missing_schema")

# Calculate score
penalty = sum(ISSUES_CONFIG[i]["score_penalty"] for i in issues_found if i in ISSUES_CONFIG)
score = max(0, 100 - penalty)

return {
    "reachable": True,
    "issues": [{"id": i, **ISSUES_CONFIG[i]} for i in issues_found],
    "score": score,
    "load_time_ms": load_time_ms,
    "html": html,
    "screenshot": None, # Will be populated by Playwright step
}

async def take_screenshot(url: str) -> bytes:
    """Take a full-page screenshot using Playwright."""
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        context = await browser.new_context(
            viewport={"width": 1280, "height": 800},
            user_agent="Mozilla/5.0 LocalAuditBot/1.0"
        )
        page = await context.new_page()
        await page.goto(url, timeout=15000, wait_until="networkidle")
        screenshot = await page.screenshot(full_page=True, type="png")
        await browser.close()
        return screenshot

```

MODULE 6 — Worker Queue (Node.js + BullMQ)

`workers/src/processors/audit.processor.ts`

typescript

```
import { Worker, Job } from 'bullmq';
import { createClient } from '@supabase/supabase-js';
import { analyzeWebsite, takeScreenshot } from '../services/scraper';
import { generateAuditReport } from '../services/ai';
import { uploadToR2 } from '../services/storage';

const supabase = createClient(
  process.env.SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_KEY!
);

export const auditWorker = new Worker('audit-queue', async (job: Job) => {
  const { auditId, tenantId, websiteUrl } = job.data;

  try {
    // Update status to running
    await supabase.from('audits').update({ status: 'running', started_at: new Date() })
      .eq('id', auditId);

    // Step 1: Analyze website
    await job.updateProgress(20);
    const analysis = await analyzeWebsite(websiteUrl);

    // Step 2: Take screenshot
    await job.updateProgress(40);
    const screenshot = await takeScreenshot(websiteUrl);
    const screenshotUrl = await uploadToR2(
      `audits/${auditId}/before.png`,
      screenshot,
      'image/png'
    );

    // Step 3: Generate AI report
    await job.updateProgress(70);
    const { data: audit } = await supabase.from('audits').select('*').eq('id', auditId);
    const report = await generateAuditReport(
      audit.business_name,
      websiteUrl,
      audit.city,
      analysis.issues
    );

    // Step 4: Save results
    await job.updateProgress(90);
    await supabase.from('audits').update({
      status: 'completed',
    });
  } catch (error) {
    // Handle error
  }
});
```

```

    issues: analysis.issues,
    score: analysis.score,
    load_time_ms: analysis.load_time_ms,
    screenshot_before: screenshotUrl,
    completed_at: new Date().toISOString(),
  }).eq('id', auditId);

// Step 5: Auto-create fix jobs if issues found
if (analysis.issues.length > 0) {
  await fixQueue.addBulk(
    analysis.issues
      .filter(issue => issue.id !== 'no_https' && issue.id !== 'site_unreachable')
      .map(issue => ({
        name: 'fix-issue',
        data: { auditId, tenantId, issueType: issue.id, websiteUrl, html: analysis.html,
        opts: { attempts: 3, backoff: { type: 'exponential', delay: 2000 } }
      })))
  );
}

await job.updateProgress(100);
return { auditId, issuesFound: analysis.issues.length, score: analysis.score };
} catch (error: any) {
  await supabase.from('audits').update({
    status: 'failed',
    error: error.message,
  }).eq('id', auditId);
  throw error;
}, {
  connection: { url: process.env.REDIS_URL },
  concurrency: 3, // 3 simultaneous audits max
});

```

MODULE 7 — Pricing Page (Next.js)

apps/web/app/(marketing)/pricing/page.tsx

typescript

```

import { headers } from 'next/headers';
import { createCheckoutSession } from '@lib/stripe';

const PLANS = [
  {
    id: 'starter',
    name: 'Starter',
    priceUsd: 29,
    credits: 50,
    stripe_price_id: process.env.NEXT_PUBLIC_STRIPE_STARTER_PRICE!,
    features: ['50 audits/month', 'AI report generation', 'Email outreach', 'Email s
  },
  {
    id: 'pro',
    name: 'Pro',
    priceUsd: 79,
    credits: 200,
    stripe_price_id: process.env.NEXT_PUBLIC_STRIPE_PRO_PRICE!,
    features: ['200 audits/month', 'AutoFix AI agent', 'PDF delivery', 'WhatsApp ale
    popular: true,
  },
  {
    id: 'agency',
    name: 'Agency',
    priceUsd: 199,
    credits: 999,
    stripe_price_id: process.env.NEXT_PUBLIC_STRIPE_AGENCY_PRICE!,
    features: ['Unlimited audits', 'White-label reports', 'API access', 'Team seats
  },
];

export default async function PricingPage() {
  // Detect visitor currency via Cloudflare header
  const headersList = headers();
  const country = headersList.get('cf-ipcountry') || 'US';
  // Use Stripe to convert price to local currency on checkout

  return (
    <div className="pricing-grid">
      {PLANS.map(plan => (
        <PricingCard key={plan.id} plan={plan} country={country} />
      ))}
    </div>
  );
}

```

MODULE 8 — Storage Service (Cloudflare R2)

apps/api/services/storage_service.py

```
python
```

```

import boto3
from botocore.config import Config
from ..core.config import get_settings

settings = get_settings()

r2_client = boto3.client(
    "s3",
    endpoint_url=f"https://{settings.CLOUDFLARE_R2_ACCOUNT_ID}.r2.cloudflarestorage.",
    aws_access_key_id=settings.CLOUDFLARE_R2_ACCESS_KEY,
    aws_secret_access_key=settings.CLOUDFLARE_R2_SECRET_KEY,
    config=Config(signature_version="s3v4"),
    region_name="auto",
)

async def upload_file(key: str, content: bytes, content_type: str) -> str:
    """Upload file to R2 and return public URL."""
    r2_client.put_object(
        Bucket=settings.CLOUDFLARE_R2_BUCKET,
        Key=key,
        Body=content,
        ContentType=content_type,
    )
    return f"{settings.CLOUDFLARE_R2_PUBLIC_URL}/{key}"

async def generate_presigned_url(key: str, expires_in: int = 3600) -> str:
    """Generate time-limited download URL for private files."""
    return r2_client.generate_presigned_url(
        "get_object",
        Params={"Bucket": settings.CLOUDFLARE_R2_BUCKET, "Key": key},
        ExpiresIn=expires_in,
    )

async def build_and_upload_delivery_zip(
    audit_id: str,
    fixed_files: list[dict],
    screenshots: dict,
    report_pdf: bytes,
) -> str:
    """Package all fixed files + report into a ZIP and upload to R2."""
    import zipfile, io

    buffer = io.BytesIO()
    with zipfile.ZipFile(buffer, "w", zipfile.ZIP_DEFLATED) as zf:
        # Add fixed HTML file
        for file in fixed_files:

```

```
        zf.writestr(file["filename"], file["content"])

# Add screenshots
if screenshots.get("before"):
    zf.writestr("screenshots/before.png", screenshots["before"])
if screenshots.get("after"):
    zf.writestr("screenshots/after.png", screenshots["after"])

# Add report PDF
zf.writestr("audit_report.pdf", report_pdf)

# Add upload instructions
zf.writestr("HOW_TO_UPLOAD.txt",
    "1. Download these files\n"
    "2. Upload them to your web hosting via cPanel File Manager or FTP\n"
    "3. Replace the existing files with the same names\n"
    "4. Clear your browser cache and check the site\n"
    "5. Reply to this email if you need help – we'll do it for you."
)

buffer.seek(0)
key = f"deliveries/{audit_id}/package.zip"
return await upload_file(key, buffer.read(), "application/zip")
```

MODULE 9 — GDPR Compliance

`apps/api/routers/gdpr.py`

python


```

from fastapi import APIRouter, Depends
from ..core.database import get_db
from ..core.security import get_current_tenant

router = APIRouter(prefix="/gdpr", tags=["GDPR"])

@router.get("/export")
async def export_data(tenant=Depends(get_current_tenant), db=Depends(get_db)):
    """Return all tenant data as JSON – GDPR Art. 20 data portability."""
    audits = await db.fetch_all("SELECT * FROM audits WHERE tenant_id = $1", tenant.id)
    users = await db.fetch_all("SELECT email, full_name, created_at FROM users WHERE tenant_id = $1", tenant.id)
    return {
        "tenant": dict(tenant),
        "users": [dict(u) for u in users],
        "audits": [dict(a) for a in audits],
        "exported_at": datetime.now().isoformat(),
    }

@router.delete("/delete-account")
async def request_deletion(tenant=Depends(get_current_tenant), db=Depends(get_db)):
    """Schedule account + data deletion – GDPR Art. 17 right to erasure.
    Actual deletion runs in 30 days (in case of billing disputes)."""
    await db.execute(
        "UPDATE tenants SET deletion_requested_at = NOW() WHERE id = $1",
        tenant.id
    )
    # Cancel Stripe subscription immediately
    if tenant.stripe_subscription_id:
        stripe.subscriptions.cancel(tenant.stripe_subscription_id)
    # Schedule deletion job for 30 days
    await enqueue_deletion(str(tenant.id), delay_days=30)
    return {"message": "Account scheduled for deletion in 30 days. Check your email"}

```



MODULE 10 — Dashboard (Next.js)

Key pages to build:

```

apps/web/app/(dashboard)/
├─ page.tsx           # Overview: credits, audits, revenue earned
├─ audits/
│   └─ page.tsx       # Audit list with status badges
│   └─ new/page.tsx   # Run audit form (URL, niche, city)
│   └─ [id]/page.tsx  # Audit detail: issues, score, fix status

```

```
└─ leads/
  └─ page.tsx          # CRM: leads, email status, conversions
└─ billing/
  └─ page.tsx          # Plan info → link to Stripe Customer Portal
└─ settings/
  └─ page.tsx          # API keys, team, notifications
└─ api-docs/
  └─ page.tsx          # Auto-generated from FastAPI OpenAPI spec
```

ENVIRONMENT VARIABLES

```
env
```

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_KEY=

# Stripe
STRIPE_SECRET_KEY=sk_live...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live...
STRIPE_WEBHOOK_SECRET=whsec...
NEXT_PUBLIC_STRIPE_STARTER_PRICE=price...
NEXT_PUBLIC_STRIPE_PRO_PRICE=price...
NEXT_PUBLIC_STRIPE_AGENCY_PRICE=price...

# AI
ANTHROPIC_API_KEY=

# Cloudflare R2
CLOUDFLARE_R2_ACCOUNT_ID=
CLOUDFLARE_R2_ACCESS_KEY=
CLOUDFLARE_R2_SECRET_KEY=
CLOUDFLARE_R2_BUCKET=localaudit-files
CLOUDFLARE_R2_PUBLIC_URL=https://files.localauditai.com

# Email
RESEND_API_KEY=

# Infrastructure
REDIS_URL=redis://...
DATABASE_URL=postgresql://...
```

PRICING STRATEGY (Global)

Plan	USD	INR	GBP	Credits	Target
Starter	\$29/mo	₹2,400/mo	£23/mo	50 audits	Solo freelancer
Pro	\$79/mo	₹6,600/mo	£63/mo	200 audits	Active agency
Agency	\$199/mo	₹16,600/mo	£159/mo	Unlimited	Large agency
One-time audit	\$4.99	₹399	£3.99	1 audit	Try before subscribe

Stripe automatically handles:

- Currency conversion at checkout based on IP
- VAT/GST via Stripe Tax (auto-configured per country)
- VAT number collection for EU B2B customers (reverse charge)
- PCI-DSS L1 — you never touch card data

COMPLIANCE CHECKLIST

- ✓ PCI-DSS — Stripe Checkout (card data never touches your server)
- ✓ GDPR — data export endpoint, deletion endpoint, cookie consent
- ✓ Data residency — Supabase EU region for EU customers
- ✓ CCPA — privacy policy + opt-out endpoint
- ✓ Terms of Service + Privacy Policy pages (required for Stripe)
- ✓ Webhook signature verification (prevents spoofed events)
- ✓ API key hashing (SHA-256, never stored in plaintext)
- ✓ Row-level security (Supabase RLS — tenants can't access each other's data)
- ✓ Rate limiting (Cloudflare + per-API-key limits)
- ✓ Audit logging (all billing events logged with idempotency keys)

DEPLOYMENT SEQUENCE

1. Set up Supabase project + run migrations
2. Create Stripe products + prices (Starter, Pro, Agency)
3. Configure Stripe Tax (enable automatic tax collection)
4. Set Stripe webhook endpoint → your domain + `/api/webhooks/stripe`
5. Set up Cloudflare R2 bucket + custom domain for file URLs
6. Deploy FastAPI to Railway (set env vars)

7. Deploy Next.js to Vercel (set env vars)
 8. Deploy worker to Railway (separate service)
 9. Set up Cloudflare in front of both domains (WAF + DDoS rules)
 10. Set up Sentry DSN in both apps
 11. Configure Grafana Cloud to pull Railway metrics
 12. Test full payment flow with Stripe test mode cards
-

BUILD ORDER (recommended)

Build in this sequence to always have something working:

1. Database schema (Supabase) + RLS policies
2. FastAPI skeleton with health check endpoint
3. Auth flow (Supabase Auth + Next.js middleware)
4. Stripe checkout + webhook handler (test mode)
5. Tenant provisioning (what happens after payment)
6. Audit service (analyzer.py + scraper.py)
7. BullMQ worker queue
8. AI audit report generation
9. AI fix generation + Claude vision validation
10. R2 storage + ZIP delivery
11. Next.js dashboard UI
12. Pricing page + billing portal
13. GDPR endpoints
14. Sentry + Grafana monitoring
15. Stripe go-live (switch to live keys)